NE

Núcleo ERP

Documentação Master: o projeto inteiro, arquivo por arquivo, e onde alterar cada coisa

ERP modular para pequenas e médias empresas, distribuído como programa instalável. Interface em React dentro do Tauri, regras de negócio em Rust e dados num arquivo SQLite no computador do cliente. Sem servidor e sem internet.

VERSÃO
0.1.0 · fase 1b

DATA
3 de novembro de 2026

REPOSITÓRIO
github.com/Igorrochalord/Esteio

Sumário

1 Visão geral
2 Como o sistema funciona
3 Mapa do repositório
4 Arquivos da raiz e CI
5 Interface (apps/desktop/src)
6 Casca nativa (src-tauri)
7 Núcleo (crates/nucleo-core)
8 Onde alterar o quê
9 Passo a passo: novo módulo
10 Banco de dados
11 Segurança e permissões
12 Referência de comandos
13 Ambiente e desenvolvimento
14 Distribuição e download
15 Backup e restauração
16 Roteiro, assinatura e pendências
17 Problemas comuns
18 Glossário

Como usar este documento. Para entender o sistema, leia os capítulos 1–3. Para achar o arquivo de uma tarefa, vá direto ao capítulo 8. Os capítulos 4–7 descrevem cada arquivo do projeto. A versão sempre atualizada fica nos arquivos `docs/*.md` do repositório.

1. Visão geral

O **Núcleo ERP** é um sistema de gestão modular. Cada empresa instala o programa no próprio computador (`.exe` no Windows); os dados ficam num arquivo local e nada depende de servidor ou internet. O sistema assume o **logo e as cores** da empresa desde a tela de entrada.

Parte	Tecnologia	Onde
Interface	React 19 + TypeScript + Vite, React Router (HashRouter)	`apps/desktop/src`
Casca nativa	Tauri v2 (Rust): janela, diálogos do sistema, ponte para o núcleo	`apps/desktop/src-tauri`
Regras de negócio	Rust: biblioteca `nucleo-core`, testável sem interface	`crates/nucleo-core`
Dados	SQLite via sqlx (WAL, chaves estrangeiras), migrações SQL	`nucleo.db` na pasta do app
Instalador	Tauri bundler: NSIS (`-setup.exe`) e MSI, gerados no CI	`.github/workflows/ci.yml`

Princípios

Uma instalação, uma empresa
A tabela `company` tem uma única linha. Não há multiempresa no mesmo arquivo.

Sem rede
A interface fala com o núcleo por `invoke` (no mesmo processo). A política de segurança (CSP) bloqueia acesso à internet.

Regra só no núcleo
A interface só mostra e coleta dados. Toda validação e permissão é decidida no Rust.

Pronto para filiais
As tabelas já guardam ids globais, filial, dispositivo e um registro de alterações para a sincronização futura.

Tudo auditado
Escritas relevantes viram histórico imutável: quem, quando e o que mudou.

Dados protegidos
Backup automático diário (7 cópias), backup manual e restauração com cópia de segurança.

Funcionalidades atuais (fase 1b)

- **Primeira abertura:** assistente que cadastra a empresa (nome, CNPJ, cores, logo) e o proprietário.
- **Entrada:** login com a marca da empresa, mostrar senha, aviso de Caps Lock, lembrar e-mail.
- **Pedir cadastro:** a pessoa cria a conta, que aguarda aprovação de um administrador.
- **Pessoas:** adicionar, mudar papel, desativar/reativar, aprovar/recusar pedidos, redefinir senha.
- **Empresa:** dados, CNPJ, cores e logo (PNG/JPEG até 2 MB).
- **Histórico:** tudo que mudou, por quem e quando, paginado.
- **Cópias de segurança:** fazer cópia, salvar em outro lugar, restaurar.

2. Como o sistema funciona

Tudo roda num único processo. A interface pede algo ao núcleo; o núcleo confere quem está logado e se pode fazer aquilo, aplica a regra, grava no banco e devolve o resultado ou um erro com mensagem pronta para o usuário.

Tela	Páginas e componentes React: mostram dados, coletam formulários, exibem mensagens.	`src/pages, src/components`
▼ companyApi.updateBranding({...})		
Serviços do front	`call(comando, args)` sobre `invoke` do Tauri; um método tipado por comando.	`src/services/api.ts, endpoints.ts`
▼ invoke("branding_update", { data })		
Comando Tauri	Camada fina: `core.require(Módulo, Ação)` e delega. **Nenhuma regra aqui.**	`src-tauri/src/commands/*.rs`
▼ company::service::update_branding(&ctx, data)		
Service	Valida, aplica a regra, abre transação, audita e faz `commit`.	`nucleo-core/src/modules/<m>/service.rs`
▼		
Repository	SQL do módulo (consultas e gravações).	`…/<m>/repository.rs`
▼		
SQLite	Tabelas, triggers de histórico de alterações e de proteção.	`migrations/*.sql → nucleo.db`

O estado em memória: `Core`

Criado ao abrir o programa (`src-tauri/src/lib.rs`), o `Core` guarda:

- a conexão com o banco (trocada na restauração de backup);
- a identidade da instalação (`device_id` , `branch_id`);
- a **sessão**: o id do usuário logado, só em memória, que some ao fechar o programa;
- uma trava de manutenção, que impede backup e restauração ao mesmo tempo.

`core.current()` relê o usuário no banco a cada comando (desativar alguém corta o acesso na hora); `core.require(m, a)` soma a checagem da matriz de permissões.

Erros

O núcleo devolve `CoreError` serializado como `{ code, message, fields }` . A tela mostra `message` , que já vem em português para o usuário final, e destaca os campos de `fields` (ex.: `owner.email`). Erro `unauthorized` faz a interface voltar ao login.

Código	Quando
`validation`	campos inválidos (vem com `fields`)
`unauthorized`	sem sessão, senha errada, usuário desativado
`forbidden`	o papel não permite a ação
`business_rule`	regra de negócio (ex.: "o dono não pode ser desativado")
`conflict`	duplicidade (e-mail já cadastrado, empresa já configurada)
`not_found`	registro inexistente
`internal`	erro inesperado; detalhe técnico só no log

Fluxo de telas

```
Primeira abertura (sem empresa) → Onboarding → já entra como proprietário
Aberturas seguintes             → Login  ⇄  Pedir cadastro (/cadastro)
Logado                          → Início · Pessoas · Empresa · Histórico · Cópias de segurança
Sessão encerrada (logout, usuário desativado, backup restaurado) → Login
```

3. Mapa do repositório

```
appIgor/
├─ .github/workflows/ci.yml     CI: lint + testes (Ubuntu) e instalador (Windows)
├─ apps/desktop/                o programa que o cliente instala
│  ├─ index.html · vite.config.ts · package.json · tsconfig*.json · eslint.config.js
│  ├─ public/                   estáticos (icon.png)
│  ├─ src/                      INTERFACE (React)
│  │  ├─ main.tsx · App.tsx · styles.css
│  │  ├─ pages/                 uma tela por arquivo
│  │  ├─ components/            peças reutilizáveis
│  │  ├─ layouts/               molduras (menu lateral, área de entrada)
│  │  ├─ hooks/                 sessão (useAuth), carregamento (useAsync)
│  │  └─ services/              chamadas ao núcleo, tipos, textos, tema
│  └─ src-tauri/                CASCA NATIVA (Rust)
│     ├─ tauri.conf.json · Cargo.toml · build.rs
│     ├─ capabilities/          o que a interface pode usar
│     ├─ icons/                 ícones do programa e do instalador
│     └─ src/  main.rs · lib.rs · commands/
├─ crates/nucleo-core/          NÚCLEO: regras de negócio + banco (Rust)
│  ├─ migrations/               0001_fundacao.sql · 0002_cadastro.sql
│  ├─ src/                      lib · db · error · permissions · audit · security · validate
│  │  ├─ modules/               auth · company · users · audit · backup
│  │  └─ bin/seed.rs            empresa de demonstração
│  └─ tests/                    testes de integração (+ common/)
├─ docs/                        documentação por tópico, adr/ e este manual
├─ Cargo.toml · Cargo.lock      workspace Rust
├─ Makefile                     atalhos: test, lint, check, seed, dev, build, manual
├─ README.md · CLAUDE.md
└─ target/ · .dev-data/         gerados localmente (fora do git)
```

Pastas geradas (fora do git)

Pasta	O que é	Pode apagar?
`target/`	compilação Rust; instaladores em `target/release/bundle/`	sim (recompila)
`.dev-data/`	banco e backups de desenvolvimento	sim (`make reset-data`)
`apps/desktop/node_modules/`	bibliotecas da interface	sim (`make install`)
`apps/desktop/dist/`	build da interface	sim
`apps/desktop/src-tauri/gen/`	esquemas gerados pelo Tauri	sim

4. Arquivos da raiz e CI

Arquivo	O que faz	Quando mexer
`Cargo.toml`	Workspace Rust: junta núcleo e app; **versão do produto**; dependências compartilhadas; dependências otimizadas mesmo em debug (argon2/SQLite rápidos); release com LTO.	subir versão; dependência Rust compartilhada
`Cargo.lock`	Versões exatas das bibliotecas Rust (build reproduzível). Gerado.	nunca à mão
`Makefile`	Atalhos: `install, test, lint, fmt, check, seed, reset-data, dev, build, web-build, manual`. Coloca `~/.cargo/bin` no PATH.	novo atalho de desenvolvimento
`README.md`	Porta de entrada: download, começar, estrutura, índice da documentação.	mudou algo visível do projeto
`CLAUDE.md`	Regras para agentes de IA (arquitetura, convenções, estado da fase).	ao fim de cada fase
`.editorconfig`	Padrão de formatação dos editores (UTF-8, LF, 2 espaços; tab no Makefile).	raramente
`.gitignore`	Ignora `target/`, `node_modules/`, `dist/`, `.dev-data/`, `src-tauri/gen/`.	nova pasta gerada
`.github/workflows/ci.yml`	Job *check* (Ubuntu): instala libs do WebKit, Rust 1.91, Node 22; roda `web-build`, `lint`, `test`. Job *windows*: gera `-setup.exe` e `.msi`, publica como artifact e, em tag `v*`, cria release em rascunho.	mudar versões de ferramentas, assinatura de código, novos alvos

Documentação (`docs/`)

Arquivo	Conteúdo
`estrutura.md`	o que cada pasta faz
`arquitetura.md`	camadas, caminho de uma ação, módulos e comandos
`desenvolvimento.md`	ambiente, comandos, testes, como criar módulo, regras de código
`banco-de-dados.md`	SQLite, migrações, regras de tabela, sync, auditoria
`seguranca-e-permissoes.md`	login, sessão, papéis, cadastro, CSP
`backup.md`	cópias automáticas/manuais e restauração
`distribuicao.md`	instalador, atalhos, publicar e baixar pelo GitHub
`assinatura.md`	pesquisa sobre planos, cobrança e licença offline
`roteiro.md`	fases e pendências
`adr/0001…, 0002…`	decisões de arquitetura (0002 substitui o SaaS pelo executável local)
`manual/manual.html`	fonte deste PDF (`make manual`)

5. Interface (`apps/desktop/src`)

Configuração do app desktop (`apps/desktop/`)

Arquivo	O que faz	Quando mexer
`package.json`	Dependências da interface (React, React Router, Tauri API, plugin de diálogo, fonte Public Sans) e scripts (`dev, build, lint, tauri`). Versão = a do produto.	nova biblioteca JS; versão
`package-lock.json`	Versões exatas das bibliotecas JS. Gerado pelo npm.	nunca à mão
`index.html`	Página que o programa carrega; título e `<div id="root">` .	raramente
`vite.config.ts`	Build da interface; porta fixa 1420 (usada pelo `tauri dev`).	raramente
`tsconfig*.json`	Regras do TypeScript (app e arquivos de configuração).	raramente
`eslint.config.js`	Regras de lint (TypeScript + hooks do React).	ajustar regra de estilo
`public/icon.png`	Ícone da aba/janela durante o desenvolvimento.	trocar identidade

Entrada e rotas

Arquivo	O que faz	Quando mexer
`main.tsx`	Carrega fontes e CSS, aplica o tema padrão e monta o `App` .	raramente
`App.tsx`	**Rotas** e quem pode abrir cada uma: `/login` , `/cadastro` , `/onboarding` (anônimos); `/` , `/usuarios` , `/empresa` , `/auditoria` , `/backup` (logados, com `RequirePermission`).	nova tela
`styles.css`	Todo o visual. Tokens no topo (`--paper, --ink, --line…`); cores da empresa via `--brand, --brand-ink, --brand-deep, --brand-deep-ink` . Seções: shell, formulários, botões, avisos, tabelas, diálogo, entrada, onboarding, responsivo.	qualquer ajuste visual

Páginas (`pages/`)

Arquivo	Tela	Usa
`LoginPage.tsx`	Entrar: marca da empresa, mostrar senha, Caps Lock, "lembrar meu e-mail" (localStorage, nunca a senha), aviso após pedido de cadastro, links.	`useAuth().login`
`RegisterPage.tsx`	Pedir cadastro: nome, e-mail, senha, confirmação → volta ao login com aviso.	`authApi.register`
`OnboardingPage.tsx`	Primeira abertura em 3 etapas (empresa, identidade visual, acesso) com prévia ao vivo; envia logo ao final.	`companyApi.onboard` , `uploadLogo`
`DashboardPage.tsx`	Início: saudação, pedidos de acesso pendentes, primeiros passos (logo, backup, pessoas).	`usersApi.list`
`UsersPage.tsx`	Pessoas: aguardando aprovação (aprovar/recusar), lista com papel, desativar/reativar, redefinir senha; diálogo de adicionar.	`usersApi.*`
`CompanyPage.tsx`	Empresa: nome/CNPJ, cores, logo.	`companyApi.*`
`AuditPage.tsx`	Histórico paginado; traduz ações e campos via `labels.ts` .	`auditApi.list`
`BackupPage.tsx`	Cópias de segurança: fazer agora, salvar em…, restaurar (só proprietário); usa diálogos nativos.	`backupApi.*` , plugin-dialog

Componentes (`components/`)

Arquivo	O que faz
`AuthProvider.tsx`	Guarda a sessão na interface: `status` (loading/setup/anonymous/authenticated), `me` , `login` , `logout` , `can()` ; aplica a marca; volta ao login quando o núcleo diz `unauthorized` .
`guards.tsx`	`RequireAuth` , `RequireAnonymous` (onboarding só sem empresa), `RequirePermission` (tela "Sem acesso").
`AuthFrame.tsx`	Moldura do login/cadastro: painel com logo, nome e cores da empresa + cartão do formulário.
`Field.tsx`	Campo com rótulo, erro, dica e botão interno opcional (`action`).
`PasswordField.tsx`	Campo de senha com "Mostrar/Ocultar" e aviso de Caps Lock.
`ColorField.tsx`	Seletor de cor + campo hexadecimal.
`BrandMark.tsx`	Logo da empresa ou iniciais sobre a cor principal.
`Dialog.tsx`	Janela modal com `<dialog>` nativo (foco preso, Esc fecha).
`Notice.tsx`	Faixa de aviso de erro ou sucesso.

Layouts, hooks e serviços

Arquivo	O que faz	Quando mexer
`layouts/AppLayout.tsx`	Menu lateral (itens filtrados por permissão), empresa, usuário, botão Sair.	novo item de menu
`layouts/AuthLayout.tsx`	Contêiner das rotas anônimas (cada página define sua moldura).	raramente
`hooks/useAuth.ts` · `authContext.ts`	Acesso ao estado de sessão / tipo desse estado.	novo dado de sessão
`hooks/useAsync.ts`	Carrega dados ao abrir a tela, com `loading` , `error` , `reload` .	raramente

Arquivo	O que faz	Quando mexer
`services/api.ts`	`call(comando, args)` sobre `invoke`; converte erros em `ApiError`; avisa sessão expirada.	raramente
`services/endpoints.ts`	Um método por comando: `authApi, companyApi, usersApi, auditApi, backupApi`. Argumentos em snake_case.	todo comando novo
`services/types.ts`	Tipos espelhando o núcleo (`User, Me, Branding, Company, AuditEntry, BackupInfo, Module…`).	campo/tipo novo; módulo novo
`services/labels.ts`	Textos: nomes de papéis, ações do histórico, nomes de campos; formatação de data e CNPJ.	ação nova na auditoria; texto
`services/theme.ts`	Cores padrão, `applyBranding()`, contraste automático do texto, iniciais.	cores padrão

6. Casca nativa (`apps/desktop/src-tauri`)

Arquivo	O que faz	Quando mexer
`tauri.conf.json`	**Nome do produto** (`Nucleo ERP`), versão, identificador (`br.com.nucleoerp.desktop`, define a pasta de dados), janela (1280×800, mín. 960×640), **CSP** (sem rede), ícones, instalador NSIS (pt-BR, por usuário).	nome, versão, tamanho de jan segurança, instalador
`Cargo.toml`	Dependências: `nucleo-core`, `tauri`, `tauri-plugin-dialog`.	novo plugin Tauri
`build.rs`	Etapa de build exigida pelo Tauri.	nunca
`capabilities/default.json`	Permissões da interface: `core:default` e diálogos (abrir, salvar, perguntar, mensagem).	usar outro recurso nativo
`icons/`	Ícones em todos os tamanhos (`icon.ico` no Windows: atalhos e instalador).	trocar ícone (`npx tauri icon arquivo.`
`src/main.rs`	Inicia o programa; sem janela de console no Windows em release.	nunca
`src/lib.rs`	Abre o `Core` na pasta de dados (ou `NUCLEO_DATA_DIR`), dispara o backup diário, registra os comandos em `generate_handler!`.	todo comando novo
`src/commands/mod.rs`	Lista dos arquivos de comando e o tipo `Res<T>`.	módulo novo
`src/commands/auth.rs`	`setup_status, login_branding, register, onboard, login, logout, me`.	—
`src/commands/company.rs`	`company_get, company_update, branding_update, logo_upload, logo_remove`.	—
`src/commands/users.rs`	`users_list, users_create, users_update, users_approve, users_reject, users_reset_password`.	—
`src/commands/audit.rs` · `backup.rs`	`audit_list` · `backup_list, backup_create, backup_restore`.	—

> Cada comando tem 2–3 linhas: `let ctx = core.require(Module::X, Action::Y).await?;` e a chamada ao service. Se você estiver escrevendo um `if` de regra de negócio num comando, ele pertence ao `service.rs`.

7. Núcleo (`crates/nucleo-core`)

Biblioteca Rust sem interface: dá para testar tudo sem abrir o programa.

Base

Arquivo	O que faz	Quando mexer
`Cargo.toml`	Dependências: sqlx (SQLite), tokio, serde, thiserror, uuid v7, chrono, argon2, base64.	nova biblioteca
`src/lib.rs`	`Core` (banco, instalação, sessão, trava), `Ctx` (banco + usuário logado), `current()` e `require()`.	raramente
`src/db.rs`	Abre o SQLite (WAL, foreign keys, busy timeout), roda migrações, `new_id()` (UUID v7), `now()`, identidade da instalação.	raramente
`src/error.rs`	`CoreError` e sua serialização; `Validator` (acumula erros por campo).	novo tipo de erro
`src/permissions.rs`	`Role`, `Module`, `Action`, **matriz** `granted()`, mapa para o menu.	módulo novo; mudar permissões
`src/audit.rs`	`snapshot`, `diff`, `record`, `record_changes` (só grava se mudou).	raramente
`src/security.rs`	Hash/verificação argon2 em thread separada; hash "falso" para tempo constante.	política de senha
`src/validate.rs`	CNPJ (dígitos verificadores), e-mail, cor hexadecimal, tamanho de texto.	nova validação comum (CPF, CEP…)
`src/modules/mod.rs`	Lista de módulos; `double_option` (diferencia campo ausente de `null`).	módulo novo
`src/bin/seed.rs`	Cria empresa de demonstração (padaria ou techparts) com logo e usuários; senha `nucleo123`.	dados de demonstração

Migrações (`migrations/`)

Arquivo	O que cria
`0001_fundacao.sql`	`installation`, `change_log`, `company`, `users`, `audit_logs`; triggers de histórico de alterações, de bloqueio de DELETE e de auditoria imutável.
`0002_cadastro.sql`	Coluna `users.pending` (autocadastro aguardando aprovação).

> **Nunca edite uma migração já distribuída.** Crie a próxima (`0003_….sql`). Elas ficam embutidas no executável e rodam ao abrir o programa.

Módulos (`src/modules/<m>/`)

Padrão: `models.rs` (structs do banco e de entrada/saída), `repository.rs` (SQL), `service.rs` (regras), `mod.rs` (declara os três).

Módulo	Arquivos e responsabilidade
`auth`	`service.rs`: `login` (tempo constante, mensagem única, aviso de pendente só com senha certa), `logout`, `me`; struct `Me` (usuário + empresa + permissões).
`company`	`models.rs`: `Company` (logo em BLOB, `logo_url` em data URL), `Branding`, entradas de onboarding/edição. `service.rs`: `status`, `public_branding`, `onboard`, `get`, `update`, `update_branding`, `upload_logo` (PNG/JPEG por assinatura, 2 MB), `remove_logo`.
`users`	`models.rs`: `User` (hash e `deleted_at` nunca serializados), `UserCreate`, `UserUpdate`, `RegisterInput`. `service.rs`: `list`, `create`, `update` (protege dono e a si mesmo), `register`, `approve`, `reject`, `reset_password`.
`audit`	`models.rs`: `AuditEntry` (com nome do autor), `AuditQuery`, `Page`. `repository.rs`: busca paginada com filtros. `service.rs`: `list` (limite 1–200).
`backup`	`service.rs`: `create`, `list`, `auto_daily` (24 h, guarda 7: `KEEP_AUTO`), `restore` (valida, guarda cópia do estado atual, troca o arquivo, desfaz se falhar).

Testes (`tests/`)

Arquivo	Garante
`common/mod.rs`	Atalhos: `setup()`, `onboarded()`, `create_member()`, `login_as()`; banco novo em pasta temporária por teste.
`foundation.rs`	Toda tabela de negócio tem colunas de sync e triggers; histórico de alterações alimentado; DELETE bloqueado; auditoria imutável; ids UUID v7.
`auth_and_permissions.rs`	Onboarding, validações, login, logout, desativação imediata, matriz de papéis, regras de pessoas, e-mail duplicado.
`registration.rs`	Autocadastro pendente, aprovação, recusa libera e-mail, só gestores aprovam, redefinição de senha, marca pública.
`company_and_backup.rs`	Edição da empresa e auditoria só do que muda, cores e logo, paginação do histórico, backup e restauração, arquivo inválido, rotação automática.

Testes de unidade ficam dentro dos próprios arquivos (`#[cfg(test)]`): permissões, validações e diff da auditoria.

8. Onde alterar o quê

Tabela de consulta rápida: a tarefa à esquerda, os arquivos à direita (caminhos a partir da raiz). `front/` = `apps/desktop/src/` · `tauri/` = `apps/desktop/src-tauri/` · `core/` = `crates/nucleo-core/`

Visual e textos

Quero...	Onde
mudar um texto de tela	a página em `front/pages/*.tsx`
mudar nome de papel, ação do histórico ou campo exibido	`front/services/labels.ts`
mudar uma mensagem de erro/validação	o `service.rs` do módulo em `core/src/modules/` (ou `core/src/error.rs` para as genéricas)
mudar cores, espaçamentos, fontes	`front/styles.css` (tokens no topo); cores da marca vêm da empresa
mudar cores padrão de empresa nova	`front/services/theme.ts` e `default_primary/secondary` em `core/src/modules/company/models.rs`
mudar a tela de login / cadastro	`front/pages/LoginPage.tsx`, `RegisterPage.tsx`, moldura em `front/components/AuthFrame.tsx`
adicionar item no menu lateral	`front/layouts/AppLayout.tsx` (lista `links`)
trocar nome do programa / atalho	`productName` em `tauri/tauri.conf.json`
trocar ícone do programa / atalho	`tauri/icons/` (gerar com `npx tauri icon arquivo.png` em `apps/desktop`)
tamanho da janela	`app.windows` em `tauri/tauri.conf.json`

Regras e dados

Quero...	Onde
mudar o que cada papel pode fazer	`granted()` em `core/src/permissions.rs` (+ teste em `tests/auth_and_permissions.rs`)
mudar regra de pessoas (quem edita quem)	`core/src/modules/users/service.rs`
tamanho mínimo de senha	`check_person` e `reset_password` em `users/service.rs` + `minLength` nas telas
desligar o autocadastro	remover a rota `/cadastro` (`front/App.tsx`), o link em `LoginPage.tsx` e o comando `register` (`tauri/src/lib.rs`)
tamanho/formatos do logo	`LOGO_MAX_BYTES` e `LOGO_TYPES` em `core/src/modules/company/service.rs`
quantas cópias automáticas guardar / frequência	`KEEP_AUTO` e `AUTO_EVERY` em `core/src/modules/backup/service.rs`
novo campo numa tabela existente	nova migração `core/migrations/000N_*.sql` → `models.rs` → `repository.rs` (COLUMNS, insert, update) → `service.rs` → `front/services/types.ts` → tela
nova tabela de negócio	migração com as regras de tabela (cap. 10); o teste `foundation.rs` confere
nova validação (CPF, CEP...)	`core/src/validate.rs` (com teste no próprio arquivo)
registrar algo no histórico	`audit::record` / `record_changes` no service; texto em `labels.ts`
dados de demonstração	`core/src/bin/seed.rs`

Telas e comandos

Quero...	Onde
nova tela	`front/pages/NovaPage.tsx` + rota em `front/App.tsx` + menu em `AppLayout.tsx`
nova ação que a tela chama	service no núcleo → comando em `tauri/src/commands/<m>.rs` → registrar em `tauri/src/lib.rs` → método em `front/services/endpoints.ts`
usar recurso nativo (arquivos, notificação...)	plugin Tauri em `tauri/Cargo.toml` + `package.json`, `.plugin(...)` em `tauri/src/lib.rs`, permissão em `tauri/capabilities/default.json`

Build, versão e publicação

Quero...	Onde
subir a versão	`Cargo.toml` (raiz), `tauri/tauri.conf.json`, `apps/desktop/package.json`: as três iguais
publicar instalador para download	`git tag vX.Y.Z && git push origin main --tags` → publicar o rascunho em Releases (cap. 14)
mudar o CI	`.github/workflows/ci.yml`
novo atalho de desenvolvimento	`Makefile`
atualizar esta documentação	`docs/*.md` e `docs/manual/manual.html`, depois `make manual`

9. Passo a passo: novo módulo

Exemplo: módulo **estoque** (`inventory`) com a tabela `products` .

1 · Migração: `core/migrations/0003_estoque.sql`

```
CREATE TABLE products (
    id          TEXT PRIMARY KEY,        -- db::new_id() (UUID v7)
    name        TEXT NOT NULL,
    price_cents INTEGER NOT NULL,        -- dinheiro em centavos, nunca float
    branch_id   TEXT NOT NULL,
    device_id   TEXT NOT NULL,
    created_at  TEXT NOT NULL,
    updated_at  TEXT NOT NULL,
    deleted_at  TEXT
);
-- copie de users: products_cl_insert, products_cl_update, products_no_delete
```

2 · Código: `core/src/modules/inventory/`

- `models.rs` : `Product` (`#[derive(Serialize, sqlx::FromRow)]`) e as entradas (`Deserialize`).
- `repository.rs` : `list` , `get` , `insert` , `update` , sempre com `deleted_at IS NULL` .
- `service.rs` : valida com `Validator` , abre `ctx.db.begin()` , grava, `audit::record` , `tx.commit()` .
- `mod.rs` e linha `pub mod inventory;` em `core/src/modules/mod.rs` .

3 · Permissão: `core/src/permissions.rs`

`Module::Inventory` (+ `ALL` e `as_str` = `"inventory"`) e o nível de cada papel em `granted()` . No front: `"inventory"` no tipo `Module` de `front/services/types.ts` .

4 · Comandos: `tauri/src/commands/inventory.rs`

```
#[tauri::command(rename_all = "snake_case")]
pub async fn products_list(core: State<'_, Core>) -> Res<Vec<Product>> {
    let ctx = core.require(Module::Inventory, Action::Read).await?;
    inventory::service::list(&ctx).await
}
```

`pub mod inventory;` em `commands/mod.rs` e `commands::inventory::products_list` no `generate_handler!` de `tauri/src/lib.rs` .

5 · Interface

- Tipo `Product` em `types.ts` ; `inventoryApi` em `endpoints.ts` .
- `pages/ProductsPage.tsx` ; rota com `RequirePermission module="inventory"` em `App.tsx` ; item em `AppLayout.tsx` .
- Textos das ações (`product.create` ...) em `labels.ts` .

6 · Testes: `core/tests/inventory.rs`

Regras, permissão por papel (quem não pode recebe `Forbidden`) e auditoria. Rode `make check` .

10. Banco de dados

Onde fica o arquivo

Situação	Pasta
Windows (instalado)	`%APPDATA%\br.com.nucleoerp.desktop\nucleo.db` (+ `backups\`)
Linux (instalado)	`~/.local/share/br.com.nucleoerp.desktop/`
Desenvolvimento	`.dev-data/` (variável `NUCLEO_DATA_DIR`)

Tabelas

Tabela	Tipo	Conteúdo
`installation`	infra	uma linha: `device_id` e `branch_id` desta instalação
`company`	negócio	uma linha: nome, CNPJ, cores, logo (BLOB + tipo + `logo_id`)
`users`	negócio	nome, e-mail (único, minúsculas), hash argon2, papel, ativo, `pending`, último acesso
`audit_logs`	infra	histórico append-only: autor, ação, entidade, antes/depois (JSON)
`change_log`	infra	fila de alterações para o sync futuro (preenchida por trigger)
`_sqlx_migrations`	infra	migrações já aplicadas (controle do sqlx)

Regras para toda tabela de negócio

- `id TEXT` com UUID v7 (único entre computadores, ordenado no tempo), nunca autoincremento.
- `branch_id`, `device_id`, `created_at`, `updated_at`, `deleted_at` (datas UTC `AAAA-MM-DDTHH:MM:SS.sssZ`).
- Triggers `<t>_cl_insert`, `<t>_cl_update` (alimentam `change_log`) e `<t>_no_delete` (exclusão só lógica).
- Consultas filtram `deleted_at IS NULL`; índices únicos com `WHERE deleted_at IS NULL`.
- Dinheiro em `INTEGER` (centavos).

> O teste `tests/foundation.rs` falha se uma tabela nova esquecer alguma dessas regras.

Auditoria

Toda escrita relevante grava autor, ação, entidade e só os campos alterados (sem `updated_at`). Campos marcados com `#[serde(skip_serializing)]` (senha, bytes do logo) nunca entram. Triggers impedem UPDATE e DELETE em `audit_logs`.

11. Segurança e permissões

Papéis

Módulo	Proprietário	Administrador	Usuário
settings (empresa, cores, logo)	manage	manage	read
users (pessoas)	manage	manage	—
audit (histórico)	manage	read	—
backup	manage (restaura)	write (faz cópia)	—

Níveis acumulam: `manage` ⊃ `write` ⊃ `read`. A interface só esconde menus; **quem decide é o núcleo** (`core.require`).

Regras de acesso

- **Senhas** com argon2; login com tempo constante e mensagem única ("E-mail ou senha inválidos").
- **Sessão** só na memória do processo; nada de token em disco; fechar o programa encerra.
- **Autocadastro** nasce inativo e pendente; só gestores aprovam (escolhendo o papel) ou recusam. O aviso "aguardando aprovação" só aparece para quem acertou a senha.
- **Redefinir senha**: por quem gerencia pessoas; ninguém redefine a própria por ali; só o proprietário mexe no proprietário; o histórico registra o fato, nunca a senha.
- O proprietário não é rebaixado nem desativado; ninguém altera o próprio papel/status; o papel `owner` só nasce no onboarding.
- **CSP**: a interface não acessa a rede; imagens só locais/ `data:` / `blob:` .
- **Logo**: PNG/JPEG conferidos pelos bytes, até 2 MB; SVG recusado (pode conter script).

> Pendências antes de distribuir: certificado de assinatura de código (evita o alerta do SmartScreen). O banco não é criptografado; se for exigido, avaliar SQLCipher.

12. Referência de comandos

Chamados pela interface via `call("nome", { args })` (argumentos em snake_case).

Comando	Argumentos	Retorna	Permissão
`setup_status`	—	`{ configured }`	livre
`login_branding`	—	`Branding | null`	livre
`register`	`data: {name, email, password}`	—	livre
`onboard`	`data: {company, owner}`	`Me`	livre (só sem empresa)
`login`	`email, password`	`Me`	livre
`logout` · `me`	—	— · `Me`	sessão
`company_get`	—	`Company`	settings read
`company_update`	`data: {name?, cnpj?}`	`Company`	settings manage
`branding_update`	`data: {primary_color?, secondary_color?}`	`Branding`	settings manage
`logo_upload` · `logo_remove`	`content` (data URL) · —	`Branding`	settings manage
`users_list`	—	`User[]`	users read
`users_create`	`data: {name, email, password, role}`	`User`	users manage
`users_update`	`user_id, data: {name?, role?, is_active?}`	`User`	users manage
`users_approve`	`user_id, role`	`User`	users manage
`users_reject`	`user_id`	—	users manage
`users_reset_password`	`user_id, password`	—	users manage
`audit_list`	`query: {limit, offset, entity?}`	`Page<AuditEntry>`	audit read
`backup_list`	—	`BackupInfo[]`	backup write
`backup_create`	`dest?`	`BackupInfo`	backup write
`backup_restore`	`path`	`BackupInfo` (cópia do estado anterior)	backup manage

13. Ambiente e desenvolvimento

Pré-requisitos

Ferramenta	Para quê
Rust 1.91+ (rustup)	núcleo e app
Node 18+ (22 recomendado)	interface e CLI do Tauri
Libs do WebKit (Linux/WSL)	compilar e abrir o app; não são necessárias para `make test`

```
sudo apt-get install -y libwebkit2gtk-4.1-dev libayatana-appindicator3-dev librsvg2-dev \
  libxdo-dev libssl-dev libdbus-1-dev pkg-config build-essential
```

No Windows nativo: Rust (rustup), Visual Studio Build Tools (C++) e Node. O WebView2 já vem no Windows 10/11.

Comandos do dia a dia

Comando	O que faz
`make install`	instala dependências da interface
`make test`	testes do núcleo (`args=nome` filtra)
`make lint`	fmt, clippy (avisos viram erro), tsc, eslint
`make check`	lint + test: obrigatório antes de commit
`make seed`	empresa demo em `.dev-data/` (`empresa=techparts` para a outra)
`make reset-data`	apaga `.dev-data/`
`make dev`	abre o programa em modo desenvolvimento (recarrega ao salvar)
`make build`	instalador da plataforma atual
`make manual`	regenera este PDF (WSL + Chrome do Windows)

Usuários do seed (senha `nucleo123`): `dono@paodourado.com.br` (proprietário), `admin@paodourado.com.br` (administrador), `caixa@paodourado.com.br` (usuário).

Convenções

- Regra de negócio só no `service.rs` ; comandos só checam permissão e delegam.
- Services devolvem `CoreError` com mensagem em pt-BR para o usuário final; validação com `Validator` e chave = caminho do campo.
- Escrita: `begin()` → alterações → auditoria → `commit()` .
- Interface: textos em pt-BR sem jargão técnico; cores da marca só por `--brand*` .
- Commits pequenos em Conventional Commits: `feat(módulo): …` , `fix` , `test` , `build` , `docs` .
- Trabalho em fases: ao fim, `make check` verde, docs atualizadas, aprovação para seguir.

14. Distribuição e download

Gerar o instalador

Onde	Como	Resultado
CI (recomendado)	push na `main` ou tag `v*`	artifact `nucleo-erp-windows` no GitHub Actions
Windows local	`make build`	`target/release/bundle/nsis/…-setup.exe` e `msi/…msi`
Linux / WSL	`make build`	`.deb` e `.AppImage`

Publicar para download (GitHub Releases)

1. Suba a versão nos três arquivos (cap. 8) e faça o commit.
2. `git tag v0.2.0 && git push origin main --tags`
3. O CI testa, gera o instalador e cria um **release em rascunho** com o `-setup.exe` e o `.msi`.
4. Em **Releases** no GitHub: revisar, escrever as novidades, **Publish release**.

> **Link para os clientes** (sempre a última versão publicada):
> `https://github.com/Igorrochalord/Esteio/releases/latest`

Instalação pelo cliente

1. Baixar `Nucleo ERP_<versão>_x64-setup.exe` no link acima.
2. Sem assinatura digital, o Windows mostra "O Windows protegeu o computador": **Mais informações → Executar assim mesmo**.
3. Seguir o instalador; na última tela, marcar **"Criar atalho na área de trabalho"**.
4. Abrir o Nucleo ERP; na primeira vez, cadastrar empresa e proprietário.

Atalhos

Instalador	Menu Iniciar	Área de trabalho
`-setup.exe` (NSIS)	sempre	caixa na última tela; automático em instalação silenciosa (`/S`); `/NS` = sem atalhos
`.msi`	sempre	sempre

No computador do cliente

O quê	Onde
Programa (NSIS, por usuário)	`%LOCALAPPDATA%\Nucleo ERP\`
Programa (MSI)	`C:\Program Files\Nucleo ERP\`
Dados	`%APPDATA%\br.com.nucleoerp.desktop\nucleo.db`
Cópias de segurança	`%APPDATA%\br.com.nucleoerp.desktop\backups\`

Programa e dados ficam separados: reinstalar ou atualizar não apaga dados.

15. Backup e restauração

Tipo	Quando	Arquivo
Automática	ao abrir, se a última tiver mais de 24 h; guarda as 7 mais novas	`auto-AAAAMMDD-HHMMSS-mmm.db`
Manual	"Fazer cópia agora"	`manual-….db`
Salvar em…	pen drive, pasta da nuvem	onde o usuário escolher
Antes de restaurar	sempre, automaticamente	`pre-restore-….db`

Como a restauração funciona

1. O arquivo é copiado para uma área temporária e conferido: SQLite íntegro, tabelas do Núcleo, versão não mais nova que o programa.
2. O estado atual é salvo em `pre-restore-….db`.
3. O banco é fechado, substituído e reaberto (migrações aplicadas).
4. Se a reabertura falhar, o estado anterior volta sozinho.
5. A sessão é encerrada (os usuários podem ser outros) e o histórico registra `backup.restore`.

> Oriente o cliente a guardar cópias **fora do computador** com frequência: as automáticas ficam no mesmo disco.

16. Roteiro, assinatura e pendências

Fase	Escopo	Estado
1	Fundação SaaS (FastAPI + Postgres), preservada na tag `v0.1-saas`	substituída
1b	Fundação local: núcleo Rust + SQLite, login, autocadastro, pessoas, empresa, histórico, backup, CI	em andamento
2	Estoque: categorias, fornecedores, produtos, movimentações imutáveis, alerta de mínimo	próxima
3 · 4 · 5	Vendas/PDV · Financeiro · Relatórios	—
6	Sincronização entre filiais (servidor + consumo do `change_log`)	—
7	Release: assinatura de código, atualização automática	—
—	Assinatura e licenciamento	proposta

Pendências da fase 1b

- Roteiro manual na interface: onboarding → login → pedir cadastro → aprovar → redefinir senha → cores/logo → histórico → backup/restauração → fechar e reabrir.
- Primeiro instalador gerado pelo CI (exige `git push`) e testado numa máquina Windows limpa.

Assinatura: resumo da proposta (`docs/assinatura.md`)

- **Planos por módulos** com limite de usuários; depois, adicional por filial quando houver sync.
- **Teste de 14–30 dias sem cartão**; ao vencer, **modo somente leitura**, nunca bloqueio dos dados.
- Mensal e anual (com desconto); cobrança com **Pix, boleto e cartão** (ex.: Asaas, com assinaturas e Pix Automático).
- Como o programa funciona sem internet: **licença assinada digitalmente** (Ed25519) com plano, limites e validade, conferida offline, renovada por um pequeno servidor que recebe o webhook de pagamento, com carência offline.
- Falta decidir: planos, preços, duração do teste e gateway.

17. Problemas comuns

Sintoma	Causa e solução
`cargo metadata … Permission denied` no `tauri dev`	O terminal não tem `~/.cargo/bin` no PATH. Use `make dev` (já corrige) ou abra um terminal novo.
Erro de `pkg-config`, `dbus` ou `webkit2gtk` ao compilar	Faltam as libs do sistema (cap. 13).
Janela em branco no WSL; avisos `libEGL` / `ZINK`	O WSL não expõe a GPU ao WebKit. `make dev` já usa `WEBKIT_DISABLE_DMABUF_RENDERER=1`.
Docker não sobe	O projeto **não usa Docker**. (Disco C: cheio derruba o Docker Desktop.)
"Este computador já tem uma empresa configurada"	Onboarding só na primeira abertura. Em dev: `make reset-data`.
Login diz "aguardando aprovação"	Autocadastro pendente: aprovar em Pessoas (proprietário/administrador).
Clippy falha no CI	Avisos viram erro; rode `make check` antes do push.
Alerta do SmartScreen no cliente	Instalador sem assinatura digital: "Mais informações → Executar assim mesmo" (definitivo: certificado de código).

18. Glossário

Termo	Significado
Tauri	Framework que transforma uma interface web num programa nativo leve, com núcleo em Rust.
`invoke` / comando	Chamada da interface para uma função Rust do programa (no mesmo processo, sem rede).
`Core` / `Ctx`	Estado do programa (banco + sessão) / contexto de um comando com o usuário logado.
Service · Repository · Model	Regras · SQL · estruturas de dados, as três camadas de cada módulo.
Migração	Script SQL numerado que cria/altera tabelas; roda sozinho ao abrir o programa.
Exclusão lógica	Marcar `deleted_at` em vez de apagar a linha.
`change_log` / outbox	Fila de alterações que a sincronização entre filiais vai consumir.
UUID v7	Identificador único global e ordenado no tempo.
Auditoria / histórico	Registro imutável de quem mudou o quê e quando.
CSP	Política de segurança do conteúdo: limita o que a interface pode carregar/acessar.
NSIS / MSI	Formatos de instalador do Windows (`-setup.exe` / `.msi`).
CI / Release	Automação no GitHub que testa e gera instaladores / página de download de uma versão.
ADR	Architecture Decision Record: registro de uma decisão de arquitetura.
WAL	Modo de gravação do SQLite que permite ler enquanto grava, com mais segurança contra queda.